

GrowthModelFit_Feb16

Stephen R. Proulx

2026-03-19

```
# --- File paths ---
growth_data_path <- "cue_data.csv"

# --- CmdStan sampling defaults ---
growth_chains <- 6
growth_parallel_chains <- 6
growth_iter_warmup <- 2000
growth_iter_sampling <- 4000
growth_adapt_delta <- 0.95
growth_max_treedepth <- 12

d_growth_raw <- read_csv(growth_data_path, show_col_types = FALSE)

d_growth_raw %>% summarise(N_raw = n())

## # A tibble: 1 x 1
##   N_raw
##   <int>
## 1    864

d_growth <- d_growth_raw %>%
  filter(!is.na(growth_rate)) %>% #remove NA data
  mutate(
    population = recode(
      population,
      "UNREL22" = "UNREL_22",
      "UNREL120" = "UNREL_120" #fix coding errors in data file: Two pops were inconsistently labelled.
    ),
    Ci = if_else(cue == "light", 1L, -1L),
    is_evo = if_else(EE == "ANC", 0L, 1L),
    evo_type = case_when(
      EE == "ANC" ~ 0L,
      EE == "REL" ~ 1L,
      EE == "UNREL" ~ 2L,
      TRUE ~ NA_integer_
    ),
    g_chr = as.character(grandmother),
    m_chr = as.character(mother),
    mat_code = paste0(g_chr, m_chr),
    mat_id = case_when(
      mat_code == "AN" ~ 1L,
```

```

    mat_code == "NN" ~ 2L,
    mat_code == "AA" ~ 3L,
    mat_code == "NA" ~ 4L,
    TRUE ~ NA_integer_
  ),
  cue_index = if_else(Ci == -1L, 1L, 2L),
  state_id = if_else(!is.na(mat_id), 2L * (mat_id - 1L) + cue_index, NA_integer_),
  log_growth = log(growth_rate),
  block_id = as.integer(factor(block))
) %>%
mutate(
  EE_seq_chr = as.character(EE_seq),
  EE_seq_id = if_else(is.na(EE_seq_chr), 0L, as.integer(factor(EE_seq_chr)))
) %>%
filter(!is.na(log_growth))

d_growth %>%
  summarise(
    N = n(),
    n_anc = sum(evo_type == 0L),
    n_rel = sum(evo_type == 1L),
    n_unrel = sum(evo_type == 2L),
    B = n_distinct(block_id),
    K_seq = max(EE_seq_id),
    Ci_vals = paste(sort(unique(Ci)), collapse = ", "),
    mat_vals = paste(sort(unique(mat_id)), collapse = ", "),
    state_range = paste(range(state_id), collapse = " .. ")
  )

```

```

## # A tibble: 1 x 9
##       N n_anc n_rel n_unrel   B K_seq Ci_vals mat_vals state_range
##   <int> <int> <int>   <int> <int> <int> <chr>   <chr>      <chr>
## 1   844   423   213     208   18    3 -1, 1    1, 2, 3, 4 1 .. 8

```

```

rel_levels <- d_growth %>%
  filter(evo_type == 1L) %>%
  distinct(population) %>%
  arrange(population) %>%
  pull(population)

unrel_levels <- d_growth %>%
  filter(evo_type == 2L) %>%
  distinct(population) %>%
  arrange(population) %>%
  pull(population)

d_growth <- d_growth %>%
  mutate(
    pop_rel_id = if_else(evo_type == 1L, match(population, rel_levels), 0L),
    pop_unrel_id = if_else(evo_type == 2L, match(population, unrel_levels), 0L)
  )

pop_check <- d_growth %>%

```

```

summarise(
  J_rel = max(pop_rel_id),
  J_unrel = max(pop_unrel_id),
  pop_ok =
    !any(evo_type != 1L & pop_rel_id != 0L) &
    !any(evo_type != 2L & pop_unrel_id != 0L)
)

#Stop if we don't have a correspondence between evo treatment and the pop ids.
stopifnot(pop_check$pop_ok)

```

A series of sanity checks, if any of these don't check out then our data processing has gone off the rails.

```

stopifnot(all(d_growth$Ci %in% c(-1L, 1L)))
stopifnot(all(d_growth$mat_id %in% 1:4))
stopifnot(all(d_growth$state_id %in% 1:8))
stopifnot(all(d_growth$is_evo %in% c(0L, 1L)))
stopifnot(all(d_growth$evo_type %in% c(0L, 1L, 2L)))
stopifnot(all(d_growth$block_id >= 1L))
stopifnot(all(d_growth$EE_seq_id >= 0L))
stopifnot(all(d_growth$pop_rel_id >= 0L))
stopifnot(all(d_growth$pop_unrel_id >= 0L))

```

Setup the stan data, basically we need a named list with each piece of data that will get passed to stan.

```

stan_data_growth <- list(
  N = nrow(d_growth),
  y = as.vector(d_growth$log_growth),
  is_evo = as.integer(d_growth$is_evo),
  evo_type = as.integer(d_growth$evo_type),
  Ci = as.integer(d_growth$Ci),
  mat_id = as.integer(d_growth$mat_id),
  state_id = as.integer(d_growth$state_id),
  use_state_anc = 1L,
  use_state = c(1L, 1L),
  B = max(d_growth$block_id),
  block_id = as.integer(d_growth$block_id),
  K_seq = max(d_growth$EE_seq_id),
  seq_id = as.integer(d_growth$EE_seq_id),
  J_rel = max(d_growth$pop_rel_id),
  pop_rel_id = as.integer(d_growth$pop_rel_id),
  J_unrel = max(d_growth$pop_unrel_id),
  pop_unrel_id = as.integer(d_growth$pop_unrel_id)
)

```

Here is stan code for just the full interaction model. This is new code and has not yet replaced the old code with all the flags and switches. We will set it up to be able to run and compare the total output.

```

stan_code_growth_interactionsonly <- "

functions {

```

```

// Center a vector so its elements sum to zero. Used to enforce sum-to-zero effects expressed
// as deviations from a baseline
vector center_sum_to_zero(vector x) {
    return x - mean(x);
}
}

data {
    // Number of observations
    int<lower=1> N;

    // log fitness, already transformed in data
    vector[N] y;

    // Indicator for whether an observation comes from an evolved population
    // 0 = ancestral, 1 = evolved
    array[N] int<lower=0, upper=1> is_evo;

    // Type of evolved treatment
    // 0 = ancestral
    // 1 = reliable evolved
    // 2 = unreliable evolved
    array[N] int<lower=0, upper=2> evo_type;

    // State index for each observation
    // Full interaction model uses 8 states that combine blue light cue and maternal/grandmaternal oxygen
    array[N] int<lower=1, upper=8> state_id;

    // Number of assay blocks
    int<lower=1> B;

    // Block identity for each observation
    array[N] int<lower=1, upper=B> block_id;

    // Number of environmental sequences used for evolved treatments
    int<lower=0> K_seq;

    // Sequence identity for each observation
    // We label the Ancestral rows can with seq_id = 0 because they are not used.
    array[N] int<lower=0, upper=K_seq> seq_id;

    // Number of reliable replicate populations
    int<lower=0> J_rel;

    // Reliable population number, within reliable numbered form 1 to j
    // all non-reliable rows are labeled with pop_rel_id = 0 because it won't be used
    array[N] int<lower=0, upper=J_rel> pop_rel_id;

    // Number of unreliable replicate populations
    int<lower=0> J_unrel;

    // Unreliable population number, within unreliable numbered form 1 to j
    // all non-unreliable rows are labeled with pop_unrel_id = 0 because it won't be used

```

```

    array[N] int<lower=0, upper=J_unrel> pop_unrel_id;
}

parameters {
    // Overall ancestral intercept
    // This is the average ancestral log fitness before adding state effects
    real alpha_anc;

    // Raw ancestral state effects for the 8 states
    // These will later made to sum to zero and scaled by a sigma term-sigma_state_anc
    vector[8] state_raw_anc;

    // Standard deviation for ancestral state effects
    real<lower=0> sigma_state_anc;

    // Evolved-treatment shifts in intercept relative to the ancestor, reliable 1 unreliable 2
    vector[2] delta_alpha;

    // treatment-specific deviations in state effects
    // These will later made to sum to zero and scaled by a sigma term, reliable 1 unreliable 2
    matrix[2, 8] delta_state_raw;

    // Standard deviations for ancestral state effects, reliable 1 unreliable 2
    vector<lower=0>[2] sigma_delta_state;

    // Raw block effects, will be scaled and sum-to-zero. These apply to all observations in this block,
    vector[B] z_block;

    // Standard deviation for block effects
    real<lower=0> sigma_block;

    // Evolutionary sequence (order of A/N) effects. Only applies to evolved populations.
    // additive model, applies to both Rel and Unrel with same parameter
    // Will be sum-to-zero and scaled
    vector[K_seq] z_seq;

    // Standard deviation for sequence effects
    real<lower=0> sigma_seq;

    // reliable population effects
    // Will be sum-to-zero and scaled
    vector[J_rel] z_pop_rel;

    // unreliable population effects
    // Will be sum-to-zero and scaled
    vector[J_unrel] z_pop_unrel;

    // standard deviation for replicate population effect
    // This is shared between reliable and unreliable populations, but
    // sum-to-zero acts within each evolutionary treatment.
    real<lower=0> sigma_pop;

    // standard deviation for the likelihood of observing each data point

```

```

    real<lower=0> sigma_y;
}

transformed parameters {
    // ancestral state effects
    // Apply sigma and sum to zero.
    vector[8] state_anc = center_sum_to_zero(sigma_state_anc * state_raw_anc);

    // Evolutionary treatment state effects- Reliable, Unreliable
    // Multiply by the appropriate sigma_delta and then enforce sum-to-zero
    matrix[2, 8] delta_state;
    for (t in 1:2) {
        delta_state[t] =
            to_row_vector(center_sum_to_zero(sigma_delta_state[t] * to_vector(delta_state_raw[t])));
    }

    // block effects, apply sigma and sum-to-zero
    vector[B] block_eff = center_sum_to_zero(sigma_block * z_block);

    // Sequence effects, multiply by sigma and sum-to-zero
    vector[K_seq] seq_eff = center_sum_to_zero(sigma_seq * z_seq);

    // Reliable population effects, multiply by sigma and sum-to-zero
    vector[J_rel] pop_rel_eff = center_sum_to_zero(sigma_pop * z_pop_rel);

    // Unreliable population effects, multiply by sigma and sum-to-zero
    vector[J_unrel] pop_unrel_eff = center_sum_to_zero(sigma_pop * z_pop_unrel);
}

model {
    // Prior for ancestral intercept
    alpha_anc ~ normal(0, 1.5);

    // Priors for ancestral state effects
    // Raw effects have standard normal prior
    // Their total scale is controlled by sigma_state_anc
    state_raw_anc ~ normal(0, 1);
    sigma_state_anc ~ exponential(1);

    // Priors for treatment-specific intercept shifts, quite broad but still regularizing
    delta_alpha ~ normal(0, 1);

    // Priors for treatment-specific state deviations
    // Raw deviations are standard normal
    // Their magnitudes are controlled by sigma_delta_state, this is common between
    // Rel and Unrel, but unique from Ancestral.
    to_vector(delta_state_raw) ~ normal(0, 1);
    sigma_delta_state ~ exponential(1);

    // Priors for block effects
    z_block ~ normal(0, 1);
    sigma_block ~ exponential(1);
}

```

```

// Priors for sequence effects
z_seq ~ normal(0, 1);
sigma_seq ~ exponential(1);

// Priors for replicate population effects
z_pop_rel ~ normal(0, 1);
z_pop_unrel ~ normal(0, 1);

// Shared prior scale for replicate population effects
sigma_pop ~ exponential(1);

// Prior for normal likelihood standard deviation
sigma_y ~ exponential(1);

// Likelihood starts here
for (n in 1:N) {
  // divide into fixed and non-fixed effects
  real mu_fixed;

  // other contributions, labelled as epsilons in print version
  real re_block;
  real re_seq;
  real re_pop;

  // For ancestral observations:
  // mu = ancestral intercept + ancestral state effect
  if (is_evo[n] == 0) {
    mu_fixed = alpha_anc + state_anc[state_id[n]];
  } else {
    // For evolved observations we have to also include shift in intercept and shift in
    // state-specific response.
    int t = evo_type[n];
    mu_fixed =
      (alpha_anc + delta_alpha[t]) +
      (state_anc[state_id[n]] + delta_state[t, state_id[n]]);
  }

  // Block effect, each observation has one.
  re_block = block_eff[block_id[n]];

  // set both the sequence and population specific effect variables at zero to start,
  // if it doesn't have a specific effect for
  // the current observation, it will add a 0 deviation to the mu value.
  re_seq = 0;
  re_pop = 0;

  // if there is a sequence effect, add it now
  if (seq_id[n] > 0) {
    re_seq = seq_eff[seq_id[n]];
  }

  if (pop_rel_id[n] > 0) {
    re_pop = pop_rel_eff[pop_rel_id[n]];
  }
}

```

```

}

// If it is an Unreliable population, set the effect
if (pop_unrel_id[n] > 0) {
  re_pop = pop_unrel_eff[pop_unrel_id[n]];
}

// Calculate the likelihood as a sampling statemnet
// sum the effects to get the expected mean for observation
y[n] ~ normal(mu_fixed + re_block + re_seq + re_pop, sigma_y);
}

}

generated quantities {
  // Pointwise log-likelihood values for model comparison
  vector[N] log_lik;

  for (n in 1:N) {
    // Recompute the same linear predictor used in the model block
    real mu_fixed;
    real re_block;
    real re_seq;
    real re_pop;
    real mu;

    // Fixed effects for ancestral or evolved observations
    if (is_evo[n] == 0) {
      mu_fixed = alpha_anc + state_anc[state_id[n]];
    } else {
      int t = evo_type[n];
      mu_fixed =
        (alpha_anc + delta_alpha[t]) +
        (state_anc[state_id[n]] + delta_state[t, state_id[n]]);
    }

    // Block effect
    re_block = block_eff[block_id[n]];

    // set both the sequence and population specific effect variables at zero to start,
    // if it doesn't have a specific effect for
    // the current observation, it will add a 0 deviation to the mu value.
    re_seq = 0;
    re_pop = 0;

    // if there is a sequence effect, add it now
    if (seq_id[n] > 0) {
      re_seq = seq_eff[seq_id[n]];
    }

    if (pop_rel_id[n] > 0) {
      re_pop = pop_rel_eff[pop_rel_id[n]];
    }
  }
}

```

```

    }

    // If it is an Unreliable population, set the effect
    if (pop_unrel_id[n] > 0) {
      re_pop = pop_unrel_eff[pop_unrel_id[n]];
    }

    // sum the effects to get the expected mean for observation
    mu = mu_fixed + re_block + re_seq + re_pop;

    // Calculate log-likelihood
    log_lik[n] = normal_lpdf(y[n] | mu, sigma_y);
  }
}

```

```

stan_file_growth_full_interaction_only <- write_stan_file(
  stan_code_growth_interactionsonly,
  dir = "stan_cache"
)

```

```

mod_growth_full_interaction_only <- cmdstan_model(
  stan_file_growth_full_interaction_only
)

```

```

stan_data_growth_full_interaction_only <- list(
  N = nrow(d_growth),
  y = as.vector(d_growth$log_growth),
  is_evo = as.integer(d_growth$is_evo),
  evo_type = as.integer(d_growth$evo_type),
  state_id = as.integer(d_growth$state_id),
  B = max(d_growth$block_id),
  block_id = as.integer(d_growth$block_id),
  K_seq = max(d_growth$EE_seq_id),
  seq_id = as.integer(d_growth$EE_seq_id),
  J_rel = max(d_growth$pop_rel_id),
  pop_rel_id = as.integer(d_growth$pop_rel_id),
  J_unrel = max(d_growth$pop_unrel_id),
  pop_unrel_id = as.integer(d_growth$pop_unrel_id)
)

```

```

fit_growth_full_interaction_only <- mod_growth_full_interaction_only$sample(
  data = stan_data_growth_full_interaction_only,
  seed = 42,
  chains = growth_chains,
  parallel_chains = growth_parallel_chains,
  iter_warmup = growth_iter_warmup,
  iter_sampling = growth_iter_sampling,
  adapt_delta = growth_adapt_delta,
  max_treedepth = growth_max_treedepth
)

```

```

## Running MCMC with 6 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 3 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 3 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 5 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 1 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 6 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 1 Iteration:  500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:  500 / 6000 [ 8%] (Warmup)
## Chain 3 Iteration:  500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 2 Iteration:  500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration:  500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration:  700 / 6000 [11%] (Warmup)
## Chain 2 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 3 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 4 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 5 Iteration:  500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 3 Iteration:  700 / 6000 [11%] (Warmup)
## Chain 1 Iteration:  800 / 6000 [13%] (Warmup)
## Chain 4 Iteration:  700 / 6000 [11%] (Warmup)
## Chain 2 Iteration:  700 / 6000 [11%] (Warmup)
## Chain 6 Iteration:  700 / 6000 [11%] (Warmup)
## Chain 3 Iteration:  800 / 6000 [13%] (Warmup)
## Chain 5 Iteration:  600 / 6000 [10%] (Warmup)
## Chain 4 Iteration:  800 / 6000 [13%] (Warmup)
## Chain 1 Iteration:  900 / 6000 [15%] (Warmup)
## Chain 2 Iteration:  800 / 6000 [13%] (Warmup)

```

```

## Chain 5 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 6 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)

```

```

## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 1 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 3 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 3 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 4 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 3 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2900 / 6000 [ 48%] (Sampling)

```

```

## Chain 3 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 4 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 2 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 4 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 2 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 1 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 3 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 4 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 5 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 2 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 6 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 4 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 2 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 6 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 4 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 3 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 2 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 5 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 4 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 6 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 6 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 5 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)

```

```

## Chain 6 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 4 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 5 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 4 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 2 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 2 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 2 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 5 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 4 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)

```

```

## Chain 5 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 3 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 5 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 6 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 6 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 6 Iteration: 4300 / 6000 [ 71%] (Sampling)

```

```

## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 5 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 1 finished in 35.0 seconds.
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 35.4 seconds.
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 finished in 36.6 seconds.
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 finished in 37.3 seconds.
## Chain 5 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 5 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 5 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 5 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 5 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 48.6 seconds.
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)

```

```
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 finished in 51.8 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 40.8 seconds.
## Total execution time: 51.9 seconds.
```

```
saveRDS(
  fit_growth_full_interaction_only$draws(),
  "stan_output/growth_full_interaction_only_draws.rds"
)
```

Setup code to compare models with/without additive and interaction effects

```
stan_code_growth_full_centered <- "
functions {
  vector center_sum_to_zero(vector x) {
    return x - mean(x);
  }
}

data {
  int<lower=1> N;
  vector[N] y;

  array[N] int<lower=0, upper=1> is_evo;
  array[N] int<lower=0, upper=2> evo_type;

  array[N] int<lower=-1, upper=1> Ci;
  array[N] int<lower=1, upper=4> mat_id;
  array[N] int<lower=1, upper=8> state_id;

  int<lower=0, upper=1> use_state_anc;
  array[2] int<lower=0, upper=1> use_state;

  int<lower=1> B;
  array[N] int<lower=1, upper=B> block_id;

  int<lower=1> K_seq;
  array[N] int<lower=0, upper=K_seq> seq_id;

  int<lower=1> J_rel;
  array[N] int<lower=0, upper=J_rel> pop_rel_id;

  int<lower=1> J_unrel;
  array[N] int<lower=0, upper=J_unrel> pop_unrel_id;
}

parameters {
  real alpha_anc;

  vector[4] mat_raw_anc;
```

```

real<lower=0> sigma_mat_anc;

real beta_cue_anc;

vector[8] state_raw_anc;
real<lower=0> sigma_state_anc;

vector[2] delta_alpha;

matrix[2, 4] delta_mat_raw;
vector<lower=0>[2] sigma_delta_mat;

vector[2] delta_beta_cue;

matrix[2, 8] delta_state_raw;
vector<lower=0>[2] sigma_delta_state;

vector[B] z_block;
real<lower=0> sigma_block;

vector[K_seq] z_seq;
real<lower=0> sigma_seq;

vector[J_rel] z_pop_rel;
vector[J_unrel] z_pop_unrel;
real<lower=0> sigma_pop;

real<lower=0> sigma_y;
}

transformed parameters {
  vector[4] mat_anc = center_sum_to_zero(sigma_mat_anc * mat_raw_anc);
  vector[8] state_anc = center_sum_to_zero(sigma_state_anc * state_raw_anc);

  matrix[2, 4] delta_mat;
  matrix[2, 8] delta_state;

  for (t in 1:2) {
    delta_mat[t] = (center_sum_to_zero(sigma_delta_mat[t] * to_vector(delta_mat_raw[t])));
    delta_state[t] = (center_sum_to_zero(sigma_delta_state[t] * to_vector(delta_state_raw[t])));
  }

  vector[B] block_eff = center_sum_to_zero(sigma_block * z_block);

  vector[K_seq] seq_eff;
  if (K_seq > 0)
    seq_eff = center_sum_to_zero(sigma_seq * z_seq);

  vector[J_rel] pop_rel_eff;
  if (J_rel > 0)
    pop_rel_eff = center_sum_to_zero(sigma_pop * z_pop_rel);

  vector[J_unrel] pop_unrel_eff;

```

```

    if (J_unrel > 0)
      pop_unrel_eff = center_sum_to_zero(sigma_pop * z_pop_unrel);
  }

model {
  alpha_anc ~ normal(0, 1.5);

  mat_raw_anc ~ normal(0, 1);
  sigma_mat_anc ~ exponential(1);

  beta_cue_anc ~ normal(0, 1);

  state_raw_anc ~ normal(0, 1);
  sigma_state_anc ~ exponential(1);

  delta_alpha ~ normal(0, 1);
  to_vector(delta_mat_raw) ~ normal(0, 1);
  sigma_delta_mat ~ exponential(1);

  delta_beta_cue ~ normal(0, 1);

  to_vector(delta_state_raw) ~ normal(0, 1);
  sigma_delta_state ~ exponential(1);

  z_block ~ normal(0, 1);
  sigma_block ~ exponential(1);

  if (K_seq > 0) {
    z_seq ~ normal(0, 1);
    sigma_seq ~ exponential(1);
  }

  if (J_rel > 0)
    z_pop_rel ~ normal(0, 1);

  if (J_unrel > 0)
    z_pop_unrel ~ normal(0, 1);

  sigma_pop ~ exponential(1);
  sigma_y ~ exponential(1);

  for (n in 1:N) {
    real mu_fixed;

    if (is_evo[n] == 0) {
      if (use_state_anc == 1) {
        mu_fixed = alpha_anc + state_anc[state_id[n]];
      } else {
        mu_fixed = alpha_anc
          + mat_anc[mat_id[n]]
          + beta_cue_anc * Ci[n];
      }
    } else {

```

```

    int t = evo_type[n];

    if (use_state[t] == 1) {
        mu_fixed = (alpha_anc + delta_alpha[t])
                  + (state_anc[state_id[n]] + delta_state[t, state_id[n]]);
    } else {
        mu_fixed = (alpha_anc + delta_alpha[t])
                  + (mat_anc[mat_id[n]] + delta_mat[t, mat_id[n]])
                  + (beta_cue_anc + delta_beta_cue[t]) * Ci[n];
    }
}

{
    real re_block = block_eff[block_id[n]];
    real re_seq = 0;
    real re_pop = 0;

    if (seq_id[n] > 0)
        re_seq = seq_eff[seq_id[n]];

    if (pop_rel_id[n] > 0)
        re_pop += pop_rel_eff[pop_rel_id[n]];

    if (pop_unrel_id[n] > 0)
        re_pop += pop_unrel_eff[pop_unrel_id[n]];

    y[n] ~ normal(mu_fixed + re_block + re_seq + re_pop, sigma_y); // likelihood of data evaluation
}
}

generated quantities {
    vector[N] log_lik;

    for (n in 1:N) {
        real mu_fixed;

        if (is_evo[n] == 0) {
            if (use_state_anc == 1) {
                mu_fixed = alpha_anc + state_anc[state_id[n]];
            } else {
                mu_fixed = alpha_anc
                          + mat_anc[mat_id[n]]
                          + beta_cue_anc * Ci[n];
            }
        } else {
            int t = evo_type[n];

            if (use_state[t] == 1) {
                mu_fixed = (alpha_anc + delta_alpha[t])
                          + (state_anc[state_id[n]] + delta_state[t, state_id[n]]);
            } else {
                mu_fixed = (alpha_anc + delta_alpha[t])

```

```

      + (mat_anc[mat_id[n]] + delta_mat[t, mat_id[n]])
      + (beta_cue_anc + delta_beta_cue[t]) * Ci[n];
    }
  }

  {
    real re_block = block_eff[block_id[n]];
    real re_seq = 0;
    real re_pop = 0;

    if (seq_id[n] > 0)
      re_seq = seq_eff[seq_id[n]];

    if (pop_rel_id[n] > 0)
      re_pop += pop_rel_eff[pop_rel_id[n]];

    if (pop_unrel_id[n] > 0)
      re_pop += pop_unrel_eff[pop_unrel_id[n]];

    log_lik[n] = normal_lpdf(y[n] | mu_fixed + re_block + re_seq + re_pop, sigma_y);
  }
}
"

stan_file_growth_full_centered <- write_stan_file(stan_code_growth_full_centered, dir = "stan_cache")

mod_growth_full_centered <- cmdstan_model(stan_file_growth_full_centered)

growth_switch_grid <- list(
  growth_additive_all = list(use_state_anc = 0L, use_state = c(0L, 0L)),
  growth_state_all    = list(use_state_anc = 1L, use_state = c(1L, 1L)),
  growth_state_anc    = list(use_state_anc = 1L, use_state = c(0L, 0L)),
  growth_state_rel    = list(use_state_anc = 0L, use_state = c(1L, 0L)),
  growth_state_unrel  = list(use_state_anc = 0L, use_state = c(0L, 1L))
)

fit_growth_switch <- lapply(names(growth_switch_grid), function(nm) {
  spec <- growth_switch_grid[[nm]]
  dat <- stan_data_growth
  dat$use_state_anc <- spec$use_state_anc
  dat$use_state <- as.integer(spec$use_state)

  mod_growth_full_centered$sample(
    data = dat,
    seed = 42,
    chains = growth_chains,
    parallel_chains = growth_parallel_chains,
    iter_warmup = growth_iter_warmup,
    iter_sampling = growth_iter_sampling,
    adapt_delta = growth_adapt_delta,
    max_treedepth = growth_max_treedepth
  )
})

```

})

```
## Running MCMC with 6 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 1 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 2 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 3 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 5 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 1 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 5 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 1 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 4 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 6 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 2 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 3 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 4 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 6 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 3 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 2 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 5 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 4 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 1 Iteration:   700 / 6000 [11%] (Warmup)
## Chain 6 Iteration:   700 / 6000 [11%] (Warmup)
## Chain 3 Iteration:   700 / 6000 [11%] (Warmup)
## Chain 2 Iteration:   700 / 6000 [11%] (Warmup)
## Chain 4 Iteration:   700 / 6000 [11%] (Warmup)
## Chain 5 Iteration:   700 / 6000 [11%] (Warmup)
## Chain 6 Iteration:   800 / 6000 [13%] (Warmup)
```

```

## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 4 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 1 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)

```

```

## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 3 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 2 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 5 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 1 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 5 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 6 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 3 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 1 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 4 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 3 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2500 / 6000 [ 41%] (Sampling)

```

```

## Chain 2 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 3 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 6 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 5 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 1 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 3 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 5 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 1 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 4 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 6 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 3 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 4 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 2 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 4 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 3 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 6 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 5 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 4 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 5 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 6 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 4 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 5 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 1 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 6 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 4 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 2 Iteration: 3300 / 6000 [ 55%] (Sampling)

```

```

## Chain 3 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 5 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 1 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 6 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 5 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 4 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 2 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 6 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 5 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 1 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 3 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 5 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 3 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 5 Iteration: 4400 / 6000 [ 73%] (Sampling)

```

```

## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 1 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 5 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 finished in 51.9 seconds.
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)

```

```

## Chain 6 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 6 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 finished in 59.9 seconds.
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 1 finished in 60.4 seconds.
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 61.4 seconds.
## Chain 6 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 61.9 seconds.
## Chain 6 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 6 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 6 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)

```

```

## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 96.9 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 65.4 seconds.
## Total execution time: 97.1 seconds.
##
## Running MCMC with 6 parallel chains...
##
## Chain 1 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 2 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 1 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 5 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 1 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration:    600 / 6000 [10%] (Warmup)
## Chain 4 Iteration:    600 / 6000 [10%] (Warmup)
## Chain 1 Iteration:    600 / 6000 [10%] (Warmup)
## Chain 2 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration:    700 / 6000 [11%] (Warmup)
## Chain 3 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 4 Iteration:    700 / 6000 [11%] (Warmup)
## Chain 1 Iteration:    700 / 6000 [11%] (Warmup)

```

```

## Chain 6 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 5 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 4 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 3 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 2 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 1 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 3 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)

```

```

## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 1 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 3 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 4 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)

```

```

## Chain 1 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 5 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 4 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 3 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 1 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 2 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 3 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 4 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 5 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 4 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 6 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 2 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 3 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 1 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 5 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 4 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 6 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 4 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 2 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 6 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 3 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 5 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 6 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 5 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 4 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 6 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 1 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 2 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 6 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)

```

```

## Chain 5 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 4 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 6 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 6 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 5 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 6 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 1 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 5 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 6 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 5 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 4 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 2 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 6 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 5 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 4 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 6 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 3 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 1 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 2 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 6 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 3 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 4300 / 6000 [ 71%] (Sampling)

```

```

## Chain 5 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 6 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 2 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 1 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 2 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 5 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 3 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 5 Iteration: 5000 / 6000 [ 83%] (Sampling)

```

```

## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 46.4 seconds.
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 1 finished in 47.0 seconds.
## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)

```

```

## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 48.1 seconds.
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 48.6 seconds.
## Chain 5 finished in 48.6 seconds.
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 3 finished in 50.9 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 48.3 seconds.
## Total execution time: 51.0 seconds.
##
## Running MCMC with 6 parallel chains...
##
## Chain 1 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:      1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 2 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:    100 / 6000 [ 1%] (Warmup)
## Chain 2 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:    200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 2 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 5 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 1 Iteration:    300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 6 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 5 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 2 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration:    400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration:    500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration:    600 / 6000 [10%] (Warmup)

```

```

## Chain 3 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 2 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 1 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 5 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 3 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 1 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 6 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 1 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 6 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 1 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)

```

```

## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 4 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 1 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2200 / 6000 [ 36%] (Sampling)

```

```

## Chain 4 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 5 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 4 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 3 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 4 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 2 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 5 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 1 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 3 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 5 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 1 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 4 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 6 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 1 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 2 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 6 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 4 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 6 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 1 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 3 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 5 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 6 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 2 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 4 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 6 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 5 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 2 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)

```

```

## Chain 6 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 5 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 6 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 5 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 1 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 2 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 6 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 5 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 6 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 2 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 6 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 4 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 5 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 6 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 3 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 1 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 6 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 6 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 2 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)

```

```

## Chain 6 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 5 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 5 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 6 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 5 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 2 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 6 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 2 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 5 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 3 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)

```

```

## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 2 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 5 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 74.7 seconds.
## Chain 3 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 finished in 75.7 seconds.
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 1 finished in 81.9 seconds.
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)

```

```

## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 finished in 85.6 seconds.
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 89.2 seconds.
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 3 finished in 95.1 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 83.7 seconds.
## Total execution time: 95.2 seconds.
##
## Running MCMC with 6 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 2 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 1 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 2 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 3 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 5 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 6 Iteration:  400 / 6000 [ 6%] (Warmup)
## Chain 1 Iteration:  400 / 6000 [ 6%] (Warmup)

```

```

## Chain 2 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 2 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 4 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 3 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 5 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 6 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 4 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 3 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 5 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 6 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 1 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 3 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 6 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)

```

```

## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)

```

[illegible]

```

## Chain 4 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 6 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 5 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 6 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 4 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 6 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 2 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 6 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 5 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 6 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 4 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 6 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 5 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 5 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 3 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 5 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 6 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)

```

```

## Chain 2 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 6 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 3 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 6 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 6 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 4 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 1 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 3 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 4 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 5 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 6 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 5 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 6 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 4 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)

```

```

## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 1 finished in 55.0 seconds.
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)

```

```

## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 finished in 57.3 seconds.
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 58.6 seconds.
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 59.5 seconds.
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 3 finished in 61.1 seconds.
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 66.9 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 59.7 seconds.
## Total execution time: 67.1 seconds.
##
## Running MCMC with 6 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:  100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 2 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:  200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 2 Iteration:  300 / 6000 [ 5%] (Warmup)
## Chain 3 Iteration:  200 / 6000 [ 3%] (Warmup)

```

```

## Chain 4 Iteration: 300 / 6000 [ 5%] (Warmup)
## Chain 5 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 1 Iteration: 300 / 6000 [ 5%] (Warmup)
## Chain 6 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration: 300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 5 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration: 400 / 6000 [ 6%] (Warmup)
## Chain 4 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 6 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 3 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration: 500 / 6000 [ 8%] (Warmup)
## Chain 5 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 6 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 3 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 3 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 6 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 1 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 1 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)

```

```

## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)

```

```

## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 3 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 6 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 5 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 2 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 4 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 5 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 3 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 6 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 2 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 5 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 3 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 1 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 6 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 2 Iteration: 2400 / 6000 [ 40%] (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 5 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 6 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%] (Sampling)
## Chain 1 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 4 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 5 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 3 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 1 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 2600 / 6000 [ 43%] (Sampling)
## Chain 6 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 4 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 5 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 3 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 1 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 2 Iteration: 2700 / 6000 [ 45%] (Sampling)
## Chain 6 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 4 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 5 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 3 Iteration: 2800 / 6000 [ 46%] (Sampling)
## Chain 2 Iteration: 2800 / 6000 [ 46%] (Sampling)

```

```

## Chain 1 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 6 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 4 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 5 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 3 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 2 Iteration: 2900 / 6000 [ 48%] (Sampling)
## Chain 1 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 6 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 5 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%] (Sampling)
## Chain 6 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 5 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 2 Iteration: 3100 / 6000 [ 51%] (Sampling)
## Chain 1 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 6 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 4 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 5 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 2 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 3 Iteration: 3200 / 6000 [ 53%] (Sampling)
## Chain 1 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 4 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 1 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 2 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 3 Iteration: 3300 / 6000 [ 55%] (Sampling)
## Chain 5 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 1 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 6 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 2 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 3 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 4 Iteration: 3400 / 6000 [ 56%] (Sampling)
## Chain 5 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 6 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%] (Sampling)
## Chain 1 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 6 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 3 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 4 Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 5 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 1 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 2 Iteration: 3700 / 6000 [ 61%] (Sampling)

```

```

## Chain 3 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 6 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 4 Iteration: 3700 / 6000 [ 61%] (Sampling)
## Chain 5 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 6 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 4 Iteration: 3800 / 6000 [ 63%] (Sampling)
## Chain 5 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 1 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 2 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 3 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 6 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 4 Iteration: 3900 / 6000 [ 65%] (Sampling)
## Chain 5 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 1 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%] (Sampling)
## Chain 5 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 2 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 1 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 3 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 6 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 4 Iteration: 4100 / 6000 [ 68%] (Sampling)
## Chain 5 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 3 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 4 Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 5 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 2 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 6 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 Iteration: 4300 / 6000 [ 71%] (Sampling)
## Chain 5 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 3 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 4 Iteration: 4400 / 6000 [ 73%] (Sampling)
## Chain 5 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 6 Iteration: 4500 / 6000 [ 75%] (Sampling)

```

```

## Chain 2 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 5 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%] (Sampling)
## Chain 3 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 6 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 5 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 4 Iteration: 4600 / 6000 [ 76%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 2 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 6 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 5 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 4700 / 6000 [ 78%] (Sampling)
## Chain 3 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 1 finished in 46.2 seconds.
## Chain 6 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 5 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 4 Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 3 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 6 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 5 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)

```

```

## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 55.4 seconds.
## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 finished in 56.6 seconds.
## Chain 5 finished in 56.6 seconds.
## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 57.5 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 57.7 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 55.0 seconds.
## Total execution time: 57.8 seconds.

```

```
fit_growth_switch <- rlang::set_names(fit_growth_switch, names(growth_switch_grid))
```

```

summarize_diag <- function(fit) {
  s <- fit$summary()

  tibble(
    n_params      = nrow(s),
    rhath_max     = max(s$rhath, na.rm = TRUE),
    rhath_gt_1_01 = sum(s$rhath > 1.01, na.rm = TRUE),
    rhath_gt_1_05 = sum(s$rhath > 1.05, na.rm = TRUE),
    ess_bulk_min  = min(s$ess_bulk, na.rm = TRUE),
    ess_tail_min  = min(s$ess_tail, na.rm = TRUE),
    n_divergent   = sum(fit$diagnostic_summary()$num_divergent, na.rm = TRUE),
    max_treedepth_hits = sum(fit$diagnostic_summary()$num_max_treedepth, na.rm = TRUE)
  ) %>%
  mutate(

```

```

    rhat_max = round(rhat_max, 3),
    ess_bulk_min = round(ess_bulk_min, 0),
    ess_tail_min = round(ess_tail_min, 0)
  )
}

summarize_model_comparison <- function(fit_list) {
  loglik_list <- lapply(fit_list, function(fit) {
    posterior::as_draws_matrix(fit$draws("log_lik"))
  })

  loo_list <- lapply(loglik_list, loo::loo)
  stacking_weights <- loo::loo_model_weights(loo_list, method = "stacking")
  loo_table <- as.data.frame(loo::loo_compare(loo_list))
  loo_table$model <- rownames(loo_table)

  loo_table %>%
    as_tibble() %>%
    mutate(weight_stacking = as.numeric(stacking_weights[model])) %>%
    select(model, elpd_diff, se_diff, elpd_loo, p_loo, looic, weight_stacking) %>%
    mutate(across(c(elpd_diff, se_diff, elpd_loo, p_loo, looic, weight_stacking), ~ round(.x, 2)))
}

```

```

growth_diag_table <- purrr::imap_dfr(
  fit_growth_switch,
  ~ summarize_diag(.x) %>% mutate(model = .y),
  .id = NULL
) %>%
  select(model, everything())

growth_switch_table <- summarize_model_comparison(fit_growth_switch)

growth_diag_table

```

```

## # A tibble: 5 x 9
##   model      n_params rhat_max rhat_gt_1_01 rhat_gt_1_05 ess_bulk_min ess_tail_min
##   <chr>      <int>    <dbl>      <int>      <int>      <dbl>      <dbl>
## 1 growth_~    1011    1.00          0          0          5624      8411
## 2 growth_~    1011    1.00          0          0          5310      8706
## 3 growth_~    1011    1.00          0          0          4805      5634
## 4 growth_~    1011    1.00          0          0          5368      8963
## 5 growth_~    1011    1.00          0          0          5907      2716
## # i 2 more variables: n_divergent <dbl>, max_treedepth_hits <int>

```

```
growth_switch_table
```

```

## # A tibble: 5 x 7
##   model      elpd_diff se_diff elpd_loo p_loo looic weight_stacking
##   <chr>      <dbl>    <dbl>    <dbl> <dbl> <dbl>      <dbl>
## 1 growth_additive_all      0      0     -580.  44.3 1160.      0.55
## 2 growth_state_rel     -0.66    2.5     -581.  47.5 1161.      0.41
## 3 growth_state_anc     -1.47    1.74     -582.  47.0 1163.      0.05

```

## 4 growth_state_unrel	-2.05	1.06	-582.	46.4	1164.	0
## 5 growth_state_all	-4	3.19	-584.	50.4	1168.	0

Model for the effect of evolutionary sequence

Added April 9, 2026

But actually do it for Unreliable too.

```
stan_code_growth_anc_rel_unrel_by_seq_state_only <- "
functions {
  vector center_sum_to_zero(vector x) {
    return x - mean(x);
  }
}

data {
  // Number of observations
  int<lower=1> N;

  // Observed log fitness
  vector[N] y;

  // 0 = Ancestral, 1 = Reliable, 2 = Unreliable
  array[N] int<lower=0, upper=2> evo_type;

  // Full 8-state index
  array[N] int<lower=1, upper=8> state_id;

  // Assay block
  int<lower=1> B;
  array[N] int<lower=1, upper=B> block_id;

  // Number of Reliable sequences
  int<lower=1> K_seq_rel;

  // Number of Unreliable sequences
  int<lower=1> K_seq_unrel;

  // Sequence ids for each observation
  // Use 0 when sequence does not apply
  array[N] int<lower=0, upper=K_seq_rel> seq_id_rel;
  array[N] int<lower=0, upper=K_seq_unrel> seq_id_unrel;

  // Total number of Reliable replicate populations
  int<lower=1> J_rel;

  // Total number of Unreliable replicate populations
  int<lower=1> J_unrel;

  // Population ids for each observation
  // Use 0 when population effect does not apply
  array[N] int<lower=0, upper=J_rel> pop_rel_id;
  array[N] int<lower=0, upper=J_unrel> pop_unrel_id;
```

```

// Sequence membership of each Reliable population
array[J_rel] int<lower=1, upper=K_seq_rel> pop_seq_of_rel;

// Sequence membership of each Unreliable population
array[J_unrel] int<lower=1, upper=K_seq_unrel> pop_seq_of_unrel;
}

parameters {
  // Ancestral mean across states
  real alpha_anc;

  // Ancestral state effects
  vector[8] state_raw_anc;
  real<lower=0> sigma_state_anc;

  // Reliable sequence-specific intercept shifts relative to Ancestral
  vector[K_seq_rel] delta_alpha_rel_seq;

  // Reliable sequence-specific state deviations relative to Ancestral
  matrix[K_seq_rel, 8] delta_state_raw_rel_seq;
  vector<lower=0>[K_seq_rel] sigma_delta_state_rel_seq;

  // Unreliable sequence-specific intercept shifts relative to Ancestral
  vector[K_seq_unrel] delta_alpha_unrel_seq;

  // Unreliable sequence-specific state deviations relative to Ancestral
  matrix[K_seq_unrel, 8] delta_state_raw_unrel_seq;
  vector<lower=0>[K_seq_unrel] sigma_delta_state_unrel_seq;

  // Block effects
  vector[B] z_block;
  real<lower=0> sigma_block;

  // Reliable replicate-population effects
  vector[J_rel] z_pop_rel;
  real<lower=0> sigma_pop_rel;

  // Unreliable replicate-population effects
  vector[J_unrel] z_pop_unrel;
  real<lower=0> sigma_pop_unrel;

  // Residual standard deviation
  real<lower=0> sigma_y;
}

transformed parameters {
  vector[8] state_anc = center_sum_to_zero(sigma_state_anc * state_raw_anc);

  matrix[K_seq_rel, 8] delta_state_rel_seq;
  matrix[K_seq_unrel, 8] delta_state_unrel_seq;

  vector[B] block_eff = center_sum_to_zero(sigma_block * z_block);

```

```

vector[J_rel] pop_rel_eff;
vector[J_unrel] pop_unrel_eff;

for (s in 1:K_seq_rel) {
  delta_state_rel_seq[s] =
    to_row_vector(
      center_sum_to_zero(
        sigma_delta_state_rel_seq[s] * to_vector(delta_state_raw_rel_seq[s])
      )
    );
}

for (s in 1:K_seq_unrel) {
  delta_state_unrel_seq[s] =
    to_row_vector(
      center_sum_to_zero(
        sigma_delta_state_unrel_seq[s] * to_vector(delta_state_raw_unrel_seq[s])
      )
    );
}

// Center Reliable population effects within sequence
for (s in 1:K_seq_rel) {
  int n_in_seq = 0;
  real mean_z = 0;

  for (j in 1:J_rel) {
    if (pop_seq_of_rel[j] == s) {
      n_in_seq += 1;
      mean_z += z_pop_rel[j];
    }
  }

  mean_z /= n_in_seq;

  for (j in 1:J_rel) {
    if (pop_seq_of_rel[j] == s) {
      pop_rel_eff[j] = sigma_pop_rel * (z_pop_rel[j] - mean_z);
    }
  }
}

// Center Unreliable population effects within sequence
for (s in 1:K_seq_unrel) {
  int n_in_seq = 0;
  real mean_z = 0;

  for (j in 1:J_unrel) {
    if (pop_seq_of_unrel[j] == s) {
      n_in_seq += 1;
      mean_z += z_pop_unrel[j];
    }
  }
}

```

```

    mean_z /= n_in_seq;

    for (j in 1:J_unrel) {
        if (pop_seq_of_unrel[j] == s) {
            pop_unrel_eff[j] = sigma_pop_unrel * (z_pop_unrel[j] - mean_z);
        }
    }
}
}

model {
    alpha_anc ~ normal(0, 1.5);

    state_raw_anc ~ normal(0, 1);
    sigma_state_anc ~ exponential(1);

    delta_alpha_rel_seq ~ normal(0, 1);
    to_vector(delta_state_raw_rel_seq) ~ normal(0, 1);
    sigma_delta_state_rel_seq ~ exponential(1);

    delta_alpha_unrel_seq ~ normal(0, 1);
    to_vector(delta_state_raw_unrel_seq) ~ normal(0, 1);
    sigma_delta_state_unrel_seq ~ exponential(1);

    z_block ~ normal(0, 1);
    sigma_block ~ exponential(1);

    z_pop_rel ~ normal(0, 1);
    sigma_pop_rel ~ exponential(1);

    z_pop_unrel ~ normal(0, 1);
    sigma_pop_unrel ~ exponential(1);

    sigma_y ~ exponential(1);

    for (n in 1:N) {
        real mu_fixed;
        real re_block;
        real re_pop;

        if (evo_type[n] == 0) {
            mu_fixed =
                alpha_anc +
                state_anc[state_id[n]];
        } else if (evo_type[n] == 1) {
            int s = seq_id_rel[n];
            mu_fixed =
                alpha_anc +
                state_anc[state_id[n]] +
                delta_alpha_rel_seq[s] +
                delta_state_rel_seq[s, state_id[n]];
        } else {
            int s = seq_id_unrel[n];

```

```

    mu_fixed =
      alpha_anc +
      state_anc[state_id[n]] +
      delta_alpha_unrel_seq[s] +
      delta_state_unrel_seq[s, state_id[n]];
  }

  re_block = block_eff[block_id[n]];
  re_pop = 0;

  if (pop_rel_id[n] > 0) {
    re_pop = pop_rel_eff[pop_rel_id[n]];
  }
  if (pop_unrel_id[n] > 0) {
    re_pop = pop_unrel_eff[pop_unrel_id[n]];
  }

  y[n] ~ normal(mu_fixed + re_block + re_pop, sigma_y);
}
}

generated quantities {
  vector[N] log_lik;
  vector[8] mu_anc_state;
  matrix[K_seq_rel, 8] mu_rel_state;
  matrix[K_seq_unrel, 8] mu_unrel_state;

  for (st in 1:8) {
    mu_anc_state[st] = alpha_anc + state_anc[st];
  }

  for (s in 1:K_seq_rel) {
    for (st in 1:8) {
      mu_rel_state[s, st] =
        alpha_anc +
        state_anc[st] +
        delta_alpha_rel_seq[s] +
        delta_state_rel_seq[s, st];
    }
  }

  for (s in 1:K_seq_unrel) {
    for (st in 1:8) {
      mu_unrel_state[s, st] =
        alpha_anc +
        state_anc[st] +
        delta_alpha_unrel_seq[s] +
        delta_state_unrel_seq[s, st];
    }
  }

  for (n in 1:N) {
    real mu_fixed;

```

```

real re_block;
real re_pop;
real mu;

if (evo_type[n] == 0) {
  mu_fixed =
    alpha_anc +
    state_anc[state_id[n]];
} else if (evo_type[n] == 1) {
  int s = seq_id_rel[n];
  mu_fixed =
    alpha_anc +
    state_anc[state_id[n]] +
    delta_alpha_rel_seq[s] +
    delta_state_rel_seq[s, state_id[n]];
} else {
  int s = seq_id_unrel[n];
  mu_fixed =
    alpha_anc +
    state_anc[state_id[n]] +
    delta_alpha_unrel_seq[s] +
    delta_state_unrel_seq[s, state_id[n]];
}

re_block = block_eff[block_id[n]];
re_pop = 0;

if (pop_rel_id[n] > 0) {
  re_pop = pop_rel_eff[pop_rel_id[n]];
}
if (pop_unrel_id[n] > 0) {
  re_pop = pop_unrel_eff[pop_unrel_id[n]];
}

mu = mu_fixed + re_block + re_pop;
log_lik[n] = normal_lpdf(y[n] | mu, sigma_y);
}
}
"

```

```

d_growth_anc_rel_unrel_seq <- d_growth %>%
  dplyr::filter(evo_type %in% c(0L, 1L, 2L))

# Re-index Reliable sequences so they run 1:K_seq_rel within this subset
rel_seq_levels <- d_growth_anc_rel_unrel_seq %>%
  dplyr::filter(evo_type == 1L) %>%
  dplyr::distinct(EE_seq_id) %>%
  dplyr::arrange(EE_seq_id) %>%
  dplyr::pull(EE_seq_id)

# Re-index Unreliable sequences so they run 1:K_seq_unrel within this subset
unrel_seq_levels <- d_growth_anc_rel_unrel_seq %>%
  dplyr::filter(evo_type == 2L) %>%

```

```

dplyr::distinct(EE_seq_id) %>%
dplyr::arrange(EE_seq_id) %>%
dplyr::pull(EE_seq_id)

d_growth_anc_rel_unrel_seq <- d_growth_anc_rel_unrel_seq %>%
dplyr::mutate(
  seq_id_rel = dplyr::if_else(
    evo_type == 1L,
    match(EE_seq_id, rel_seq_levels),
    0L
  ),
  seq_id_unrel = dplyr::if_else(
    evo_type == 2L,
    match(EE_seq_id, unrel_seq_levels),
    0L
  )
)

# Re-index Reliable populations within this subset
rel_pop_levels_seq <- d_growth_anc_rel_unrel_seq %>%
dplyr::filter(evo_type == 1L) %>%
dplyr::distinct(population) %>%
dplyr::arrange(population) %>%
dplyr::pull(population)

# Re-index Unreliable populations within this subset
unrel_pop_levels_seq <- d_growth_anc_rel_unrel_seq %>%
dplyr::filter(evo_type == 2L) %>%
dplyr::distinct(population) %>%
dplyr::arrange(population) %>%
dplyr::pull(population)

d_growth_anc_rel_unrel_seq <- d_growth_anc_rel_unrel_seq %>%
dplyr::mutate(
  pop_rel_id = dplyr::if_else(
    evo_type == 1L,
    match(population, rel_pop_levels_seq),
    0L
  ),
  pop_unrel_id = dplyr::if_else(
    evo_type == 2L,
    match(population, unrel_pop_levels_seq),
    0L
  )
)

# Map each Reliable population to its sequence
pop_seq_of_rel <- d_growth_anc_rel_unrel_seq %>%
dplyr::filter(evo_type == 1L) %>%
dplyr::distinct(pop_rel_id, seq_id_rel) %>%
dplyr::arrange(pop_rel_id) %>%
dplyr::pull(seq_id_rel)

```

```

# Map each Unreliable population to its sequence
pop_seq_of_unrel <- d_growth_anc_rel_unrel_seq %>%
  dplyr::filter(evo_type == 2L) %>%
  dplyr::distinct(pop_unrel_id, seq_id_unrel) %>%
  dplyr::arrange(pop_unrel_id) %>%
  dplyr::pull(seq_id_unrel)

stan_data_growth_anc_rel_unrel_by_seq_state_only <- list(
  N = nrow(d_growth_anc_rel_unrel_seq),
  y = as.vector(d_growth_anc_rel_unrel_seq$log_growth),
  evo_type = as.integer(d_growth_anc_rel_unrel_seq$evo_type),
  state_id = as.integer(d_growth_anc_rel_unrel_seq$state_id),
  B = max(d_growth_anc_rel_unrel_seq$block_id),
  block_id = as.integer(d_growth_anc_rel_unrel_seq$block_id),
  K_seq_rel = length(rel_seq_levels),
  K_seq_unrel = length(unrel_seq_levels),
  seq_id_rel = as.integer(d_growth_anc_rel_unrel_seq$seq_id_rel),
  seq_id_unrel = as.integer(d_growth_anc_rel_unrel_seq$seq_id_unrel),
  J_rel = length(rel_pop_levels_seq),
  J_unrel = length(unrel_pop_levels_seq),
  pop_rel_id = as.integer(d_growth_anc_rel_unrel_seq$pop_rel_id),
  pop_unrel_id = as.integer(d_growth_anc_rel_unrel_seq$pop_unrel_id),
  pop_seq_of_rel = as.integer(pop_seq_of_rel),
  pop_seq_of_unrel = as.integer(pop_seq_of_unrel)
)

str(stan_data_growth_anc_rel_unrel_by_seq_state_only, max.level = 1)

```

```

## List of 16
## $ N : int 844
## $ y : num [1:844] 1.101 0.871 0.113 0.913 1.055 ...
## $ evo_type : int [1:844] 0 0 0 0 0 0 0 0 0 ...
## $ state_id : int [1:844] 6 6 6 5 5 5 2 2 2 1 ...
## $ B : int 18
## $ block_id : int [1:844] 1 1 1 1 1 1 1 1 1 1 ...
## $ K_seq_rel : int 3
## $ K_seq_unrel : int 3
## $ seq_id_rel : int [1:844] 0 0 0 0 0 0 0 0 0 ...
## $ seq_id_unrel : int [1:844] 0 0 0 0 0 0 0 0 0 ...
## $ J_rel : int 9
## $ J_unrel : int 9
## $ pop_rel_id : int [1:844] 0 0 0 0 0 0 0 0 0 ...
## $ pop_unrel_id : int [1:844] 0 0 0 0 0 0 0 0 0 ...
## $ pop_seq_of_rel : int [1:9] 2 1 2 2 3 3 3 1 1
## $ pop_seq_of_unrel: int [1:9] 1 3 1 3 3 2 1 2 2

```

```

stan_file_growth_anc_rel_unrel_by_seq_state_only <- write_stan_file(
  stan_code_growth_anc_rel_unrel_by_seq_state_only,
  dir = "stan_cache"
)

mod_growth_anc_rel_unrel_by_seq_state_only <- cmdstan_model(
  stan_file_growth_anc_rel_unrel_by_seq_state_only
)

```

```
)

fit_growth_anc_rel_unrel_by_seq_state_only <- mod_growth_anc_rel_unrel_by_seq_state_only$sample(
  data = stan_data_growth_anc_rel_unrel_by_seq_state_only,
  seed = 42,
  chains = growth_chains,
  parallel_chains = growth_parallel_chains,
  iter_warmup = growth_iter_warmup,
  iter_sampling = growth_iter_sampling,
  adapt_delta = growth_adapt_delta,
  max_treedepth = growth_max_treedepth
)
```

```
## Running MCMC with 6 parallel chains...
```

```
##
## Chain 1 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 5 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 6 Iteration:    1 / 6000 [ 0%] (Warmup)
## Chain 1 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 3 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 4 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 5 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 6 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 2 Iteration:   100 / 6000 [ 1%] (Warmup)
## Chain 1 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 5 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 6 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 1 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 4 Iteration:   200 / 6000 [ 3%] (Warmup)
## Chain 3 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 5 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 1 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 6 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 2 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 4 Iteration:   300 / 6000 [ 5%] (Warmup)
## Chain 3 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 1 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 4 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 5 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 2 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 6 Iteration:   400 / 6000 [ 6%] (Warmup)
## Chain 3 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 1 Iteration:   600 / 6000 [10%] (Warmup)
## Chain 5 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 4 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 6 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 2 Iteration:   500 / 6000 [ 8%] (Warmup)
## Chain 3 Iteration:   600 / 6000 [10%] (Warmup)
```

```

## Chain 1 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 5 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 6 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 2 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 3 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 600 / 6000 [ 10%] (Warmup)
## Chain 5 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 1 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 6 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 2 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 4 Iteration: 700 / 6000 [ 11%] (Warmup)
## Chain 3 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 5 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 1 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 2 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 4 Iteration: 800 / 6000 [ 13%] (Warmup)
## Chain 3 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 5 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 6 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 2 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 4 Iteration: 900 / 6000 [ 15%] (Warmup)
## Chain 1 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 5 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 6 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%] (Warmup)
## Chain 1 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 3 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 5 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 6 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 2 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 4 Iteration: 1100 / 6000 [ 18%] (Warmup)
## Chain 1 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 5 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 6 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 2 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 4 Iteration: 1200 / 6000 [ 20%] (Warmup)
## Chain 1 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 6 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 2 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 3 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 1 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1300 / 6000 [ 21%] (Warmup)
## Chain 5 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 6 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 2 Iteration: 1400 / 6000 [ 23%] (Warmup)
## Chain 3 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 4 Iteration: 1400 / 6000 [ 23%] (Warmup)

```

```

## Chain 5 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 6 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 1 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 4 Iteration: 1500 / 6000 [ 25%] (Warmup)
## Chain 5 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 6 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 2 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 3 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 4 Iteration: 1600 / 6000 [ 26%] (Warmup)
## Chain 5 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 1 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 6 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 2 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 3 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 4 Iteration: 1700 / 6000 [ 28%] (Warmup)
## Chain 5 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 1 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 6 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 2 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 3 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 1800 / 6000 [ 30%] (Warmup)
## Chain 5 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 6 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 1 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 1 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 2 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 4 Iteration: 1900 / 6000 [ 31%] (Warmup)
## Chain 5 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 5 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 3 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 6 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 1 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 2 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 5 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%] (Warmup)
## Chain 4 Iteration: 2001 / 6000 [ 33%] (Sampling)
## Chain 6 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 1 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 2 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 3 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 4 Iteration: 2100 / 6000 [ 35%] (Sampling)
## Chain 5 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 6 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 1 Iteration: 2300 / 6000 [ 38%] (Sampling)
## Chain 2 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 4 Iteration: 2200 / 6000 [ 36%] (Sampling)
## Chain 5 Iteration: 2300 / 6000 [ 38%] (Sampling)

```

[illegible]

[illegible]

Chain 1 Iteration: 4100 / 6000 [68%] (Sampling)
Chain 4 Iteration: 4000 / 6000 [66%] (Sampling)
Chain 5 Iteration: 4300 / 6000 [71%] (Sampling)
Chain 6 Iteration: 4200 / 6000 [70%] (Sampling)
Chain 3 Iteration: 4100 / 6000 [68%] (Sampling)
Chain 2 Iteration: 4100 / 6000 [68%] (Sampling)
Chain 1 Iteration: 4200 / 6000 [70%] (Sampling)
Chain 4 Iteration: 4100 / 6000 [68%] (Sampling)
Chain 5 Iteration: 4400 / 6000 [73%] (Sampling)
Chain 6 Iteration: 4300 / 6000 [71%] (Sampling)
Chain 3 Iteration: 4200 / 6000 [70%] (Sampling)
Chain 2 Iteration: 4200 / 6000 [70%] (Sampling)
Chain 1 Iteration: 4300 / 6000 [71%] (Sampling)
Chain 5 Iteration: 4500 / 6000 [75%] (Sampling)
Chain 4 Iteration: 4200 / 6000 [70%] (Sampling)
Chain 6 Iteration: 4400 / 6000 [73%] (Sampling)
Chain 3 Iteration: 4300 / 6000 [71%] (Sampling)
Chain 1 Iteration: 4400 / 6000 [73%] (Sampling)
Chain 2 Iteration: 4300 / 6000 [71%] (Sampling)
Chain 5 Iteration: 4600 / 6000 [76%] (Sampling)
Chain 4 Iteration: 4300 / 6000 [71%] (Sampling)
Chain 6 Iteration: 4500 / 6000 [75%] (Sampling)
Chain 3 Iteration: 4400 / 6000 [73%] (Sampling)
Chain 2 Iteration: 4400 / 6000 [73%] (Sampling)
Chain 5 Iteration: 4700 / 6000 [78%] (Sampling)
Chain 1 Iteration: 4500 / 6000 [75%] (Sampling)
Chain 4 Iteration: 4400 / 6000 [73%] (Sampling)
Chain 6 Iteration: 4600 / 6000 [76%] (Sampling)
Chain 3 Iteration: 4500 / 6000 [75%] (Sampling)
Chain 2 Iteration: 4500 / 6000 [75%] (Sampling)
Chain 5 Iteration: 4800 / 6000 [80%] (Sampling)
Chain 1 Iteration: 4600 / 6000 [76%] (Sampling)
Chain 6 Iteration: 4700 / 6000 [78%] (Sampling)
Chain 3 Iteration: 4600 / 6000 [76%] (Sampling)
Chain 4 Iteration: 4500 / 6000 [75%] (Sampling)
Chain 5 Iteration: 4900 / 6000 [81%] (Sampling)
Chain 1 Iteration: 4700 / 6000 [78%] (Sampling)
Chain 2 Iteration: 4600 / 6000 [76%] (Sampling)
Chain 6 Iteration: 4800 / 6000 [80%] (Sampling)
Chain 4 Iteration: 4600 / 6000 [76%] (Sampling)
Chain 3 Iteration: 4700 / 6000 [78%] (Sampling)
Chain 5 Iteration: 5000 / 6000 [83%] (Sampling)
Chain 2 Iteration: 4700 / 6000 [78%] (Sampling)
Chain 6 Iteration: 4900 / 6000 [81%] (Sampling)
Chain 1 Iteration: 4800 / 6000 [80%] (Sampling)
Chain 4 Iteration: 4700 / 6000 [78%] (Sampling)
Chain 3 Iteration: 4800 / 6000 [80%] (Sampling)
Chain 5 Iteration: 5100 / 6000 [85%] (Sampling)
Chain 2 Iteration: 4800 / 6000 [80%] (Sampling)
Chain 6 Iteration: 5000 / 6000 [83%] (Sampling)
Chain 1 Iteration: 4900 / 6000 [81%] (Sampling)
Chain 4 Iteration: 4800 / 6000 [80%] (Sampling)
Chain 3 Iteration: 4900 / 6000 [81%] (Sampling)
Chain 5 Iteration: 5200 / 6000 [86%] (Sampling)

```

## Chain 2 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 6 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 4 Iteration: 4900 / 6000 [ 81%] (Sampling)
## Chain 5 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 5 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 6 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 1 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%] (Sampling)
## Chain 3 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 2 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 5 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 6 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 1 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 4 Iteration: 5100 / 6000 [ 85%] (Sampling)
## Chain 3 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 2 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 5 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 6 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 4 Iteration: 5200 / 6000 [ 86%] (Sampling)
## Chain 3 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 2 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 5 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 4 Iteration: 5300 / 6000 [ 88%] (Sampling)
## Chain 3 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 5 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 6 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 2 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 4 Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 5 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 6 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 1 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%] (Sampling)
## Chain 2 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 5 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 5 finished in 20.7 seconds.
## Chain 1 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 4 Iteration: 5600 / 6000 [ 93%] (Sampling)
## Chain 3 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 6 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 2 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 1 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 4 Iteration: 5700 / 6000 [ 95%] (Sampling)
## Chain 3 Iteration: 5800 / 6000 [ 96%] (Sampling)

```

```

## Chain 6 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 6 finished in 21.3 seconds.
## Chain 2 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 1 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 3 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 4 Iteration: 5800 / 6000 [ 96%] (Sampling)
## Chain 2 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 Iteration: 5900 / 6000 [ 98%] (Sampling)
## Chain 1 finished in 22.1 seconds.
## Chain 3 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 3 finished in 22.2 seconds.
## Chain 2 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 2 finished in 22.3 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 4 finished in 22.4 seconds.
##
## All 6 chains finished successfully.
## Mean chain execution time: 21.8 seconds.
## Total execution time: 22.6 seconds.

```

```

saveRDS(
  fit_growth_anc_rel_unrel_by_seq_state_only$draws(),
  "stan_output/growth_anc_rel_unrel_by_seq_state_only_draws.rds"
)

```